



Chapter 4 Memory Management

Configuration of a single partition

- Divide the memory space into two pieces, one for the operating system and the other for the process
- You can use the hardware's **base registers** to prevent processes from accessing the operating system's memory address
- If the program generates an address that is smaller than the address in the **base register**, an **exception break** with an addressing error is generated

Figure 4-1.a Legitimate Logical Address

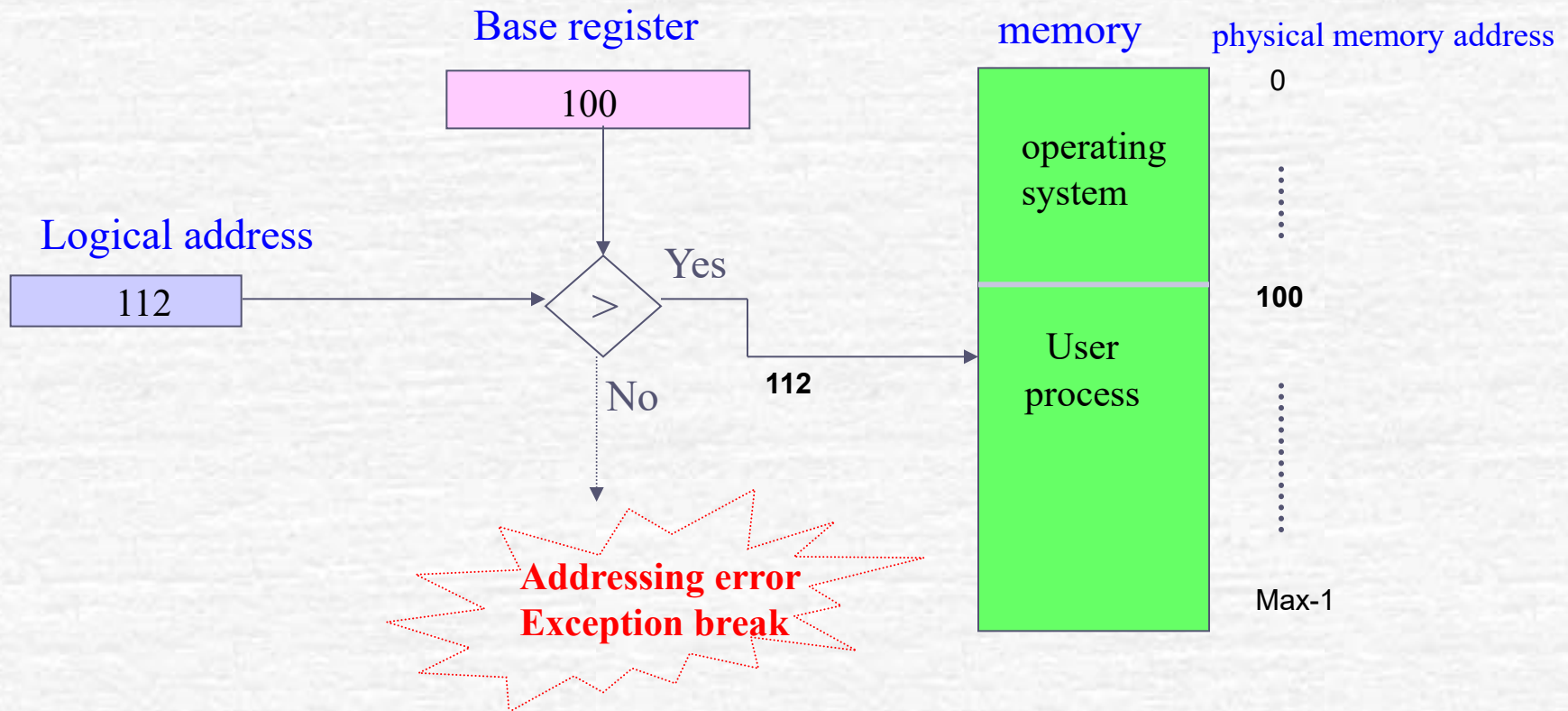
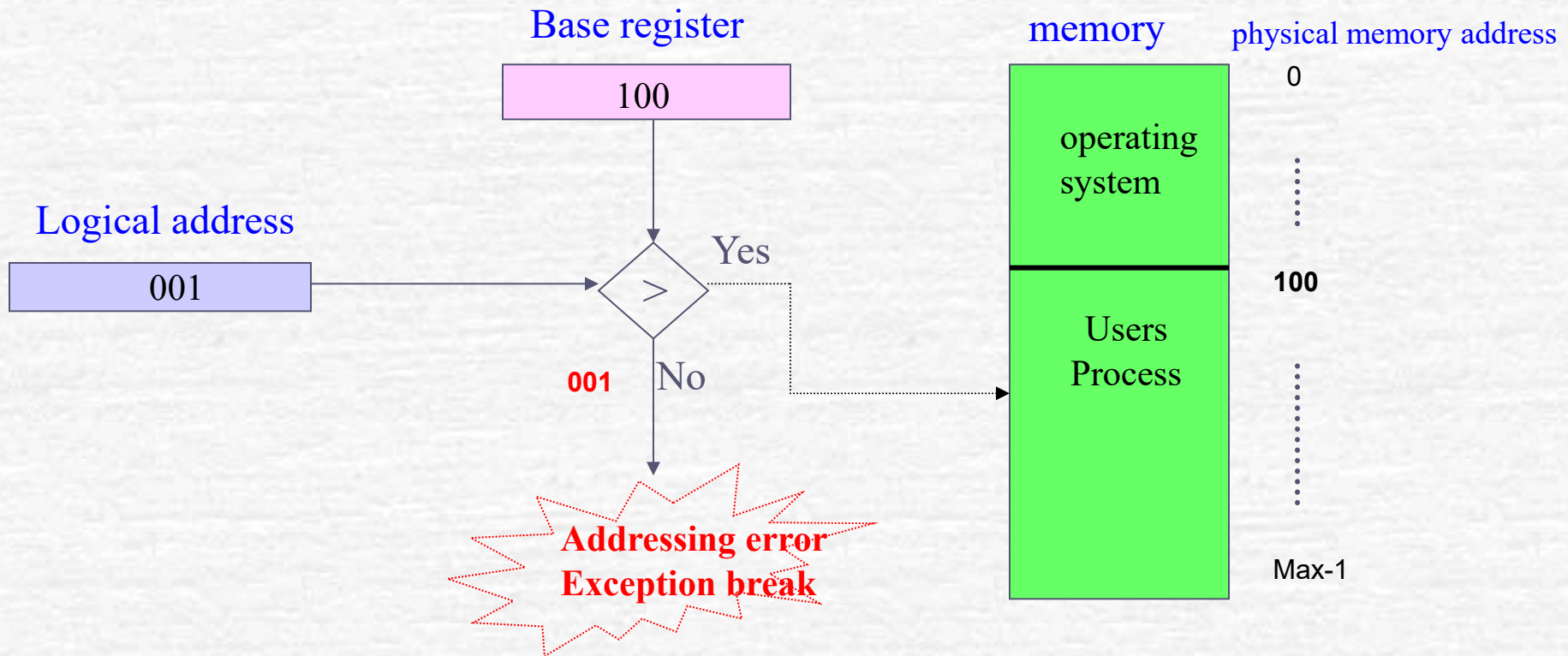


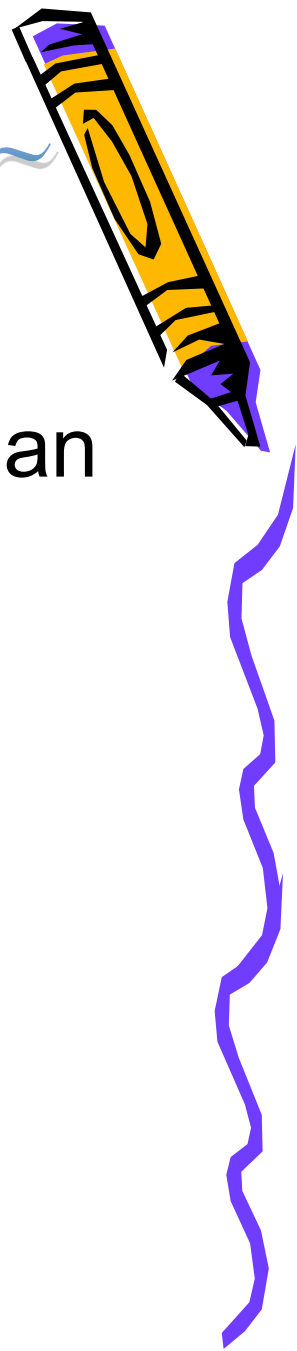
Figure 4-1.b The memory address accessed to the operating system



Classroom exercises

- Suppose the operating system is in a contiguous section of physical memory starting at 0 and is 256 KB long
 - What should be the minimum value of the base register?
 - Which of the following addresses is a legal itinerary address?
 1. 300K
 2. 25K
 3. 300





Practice Answers

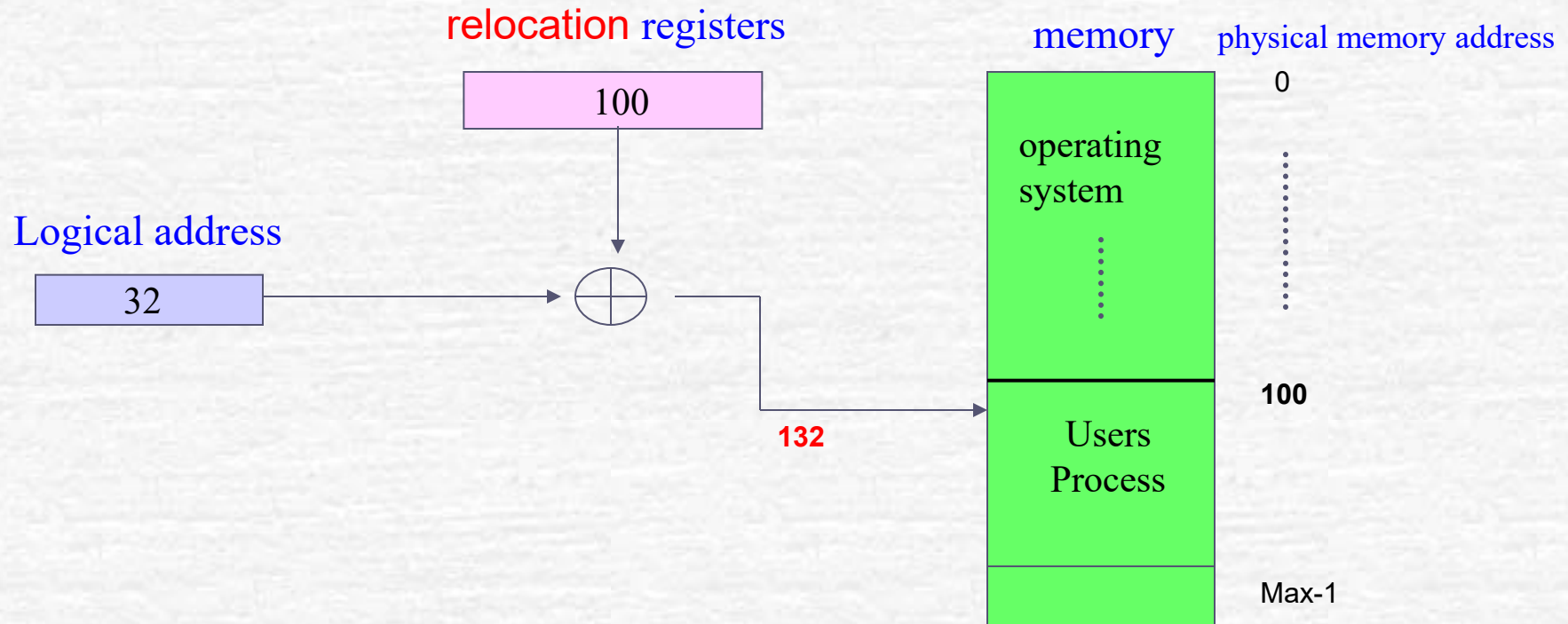
- Value of the base register: greater than or equal to 256K
- Process :
 1. 300K-Legal
 2. 25K-Not legal
 3. 600-Not legal



Relocation Register

- Another way to protect is to use a **relocation register**, which is used to store the start address of the process
- When compiling, configure the program from 0
 - **Logical address** : The program seems to run in a large continuous logical address space starting from 0
 - **Physical address** : The memory management hardware is responsible for converting logical addresses into real addresses in physical memory

Figure 4-2 Memory address conversion using readdress registers



Loading of the program

- **Absolute code** : When the program is compiled, it is predetermined where the program will be executed in physical memory, and the logical address of the compiled code will be equal to the physical address
 - The disadvantage is that the code can only be placed in a fixed location in memory, and when multiple processes want to share memory, it may interfere with each other
- **Re-addressable code** : The address is configured from 0 at compile time, and then it is decided where to put it when loading
 - The **physical address** of the process must be obtained by adding the contents of the relocation register through the **logical address**
 - If you need to change the location of the process in physical memory, simply change the value of the relocation register after the migration process

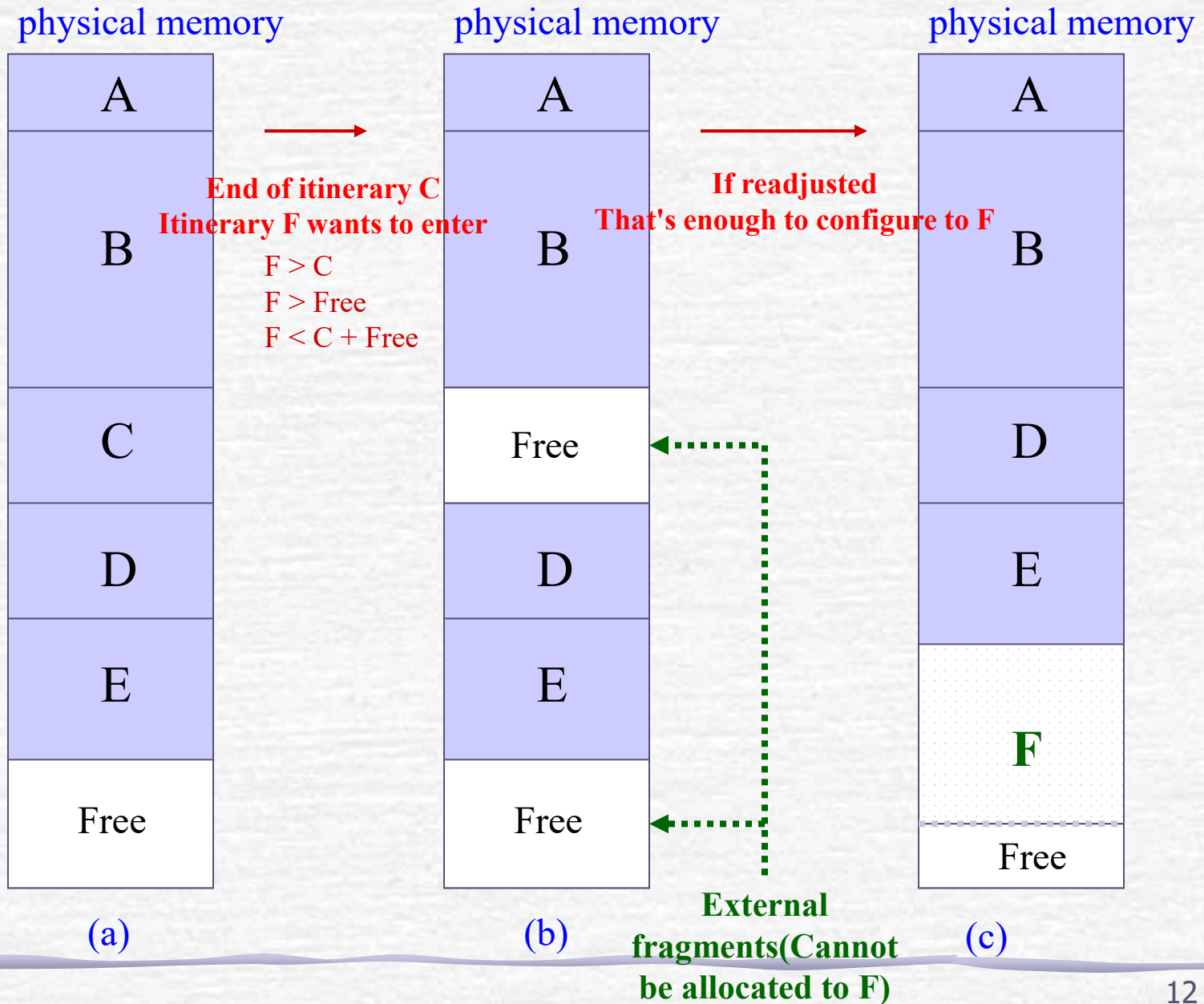
Memory allocation for multiple trips - fixed length

- To cut memory into partitions of the same size, the operating system only needs to remember which partition each process is assigned to - **the simplest way to allocate multi-strokes**
- Because the length of the partition is the same, the value of the length register can be set when **booting**
- When a process gains CPU control, the operating system resets the contents of the readdress register to the partition where the process resides
- If you partition the memory into partitions of the **same size**, there will be a lot of space left in **some areas**, called **internal fragmentation**
- You can divide memory into several fixed-length partitions of **different sizes**, and allocate different sizes of travel into partitions of different lengths
- When it is the turn of a process to use the CPU, in addition to setting the **relocation register**, the operating system must set the value of the length register to the length of the partition in which the process is located

Multi-partition of variable length

- Another way to reduce internal fragmentation is not to partition the memory in advance, but to dynamically divide the memory based on the current memory usage and the length of the process to be loaded
- It can minimize **internal debris**, but it can cause **external debris**
- If the partition configuration is not properly chosen, it may aggravate the problem of external fragmentation
- The **aggregation method** can be used to remove each trip to gather these fragments into a complete large continuous space

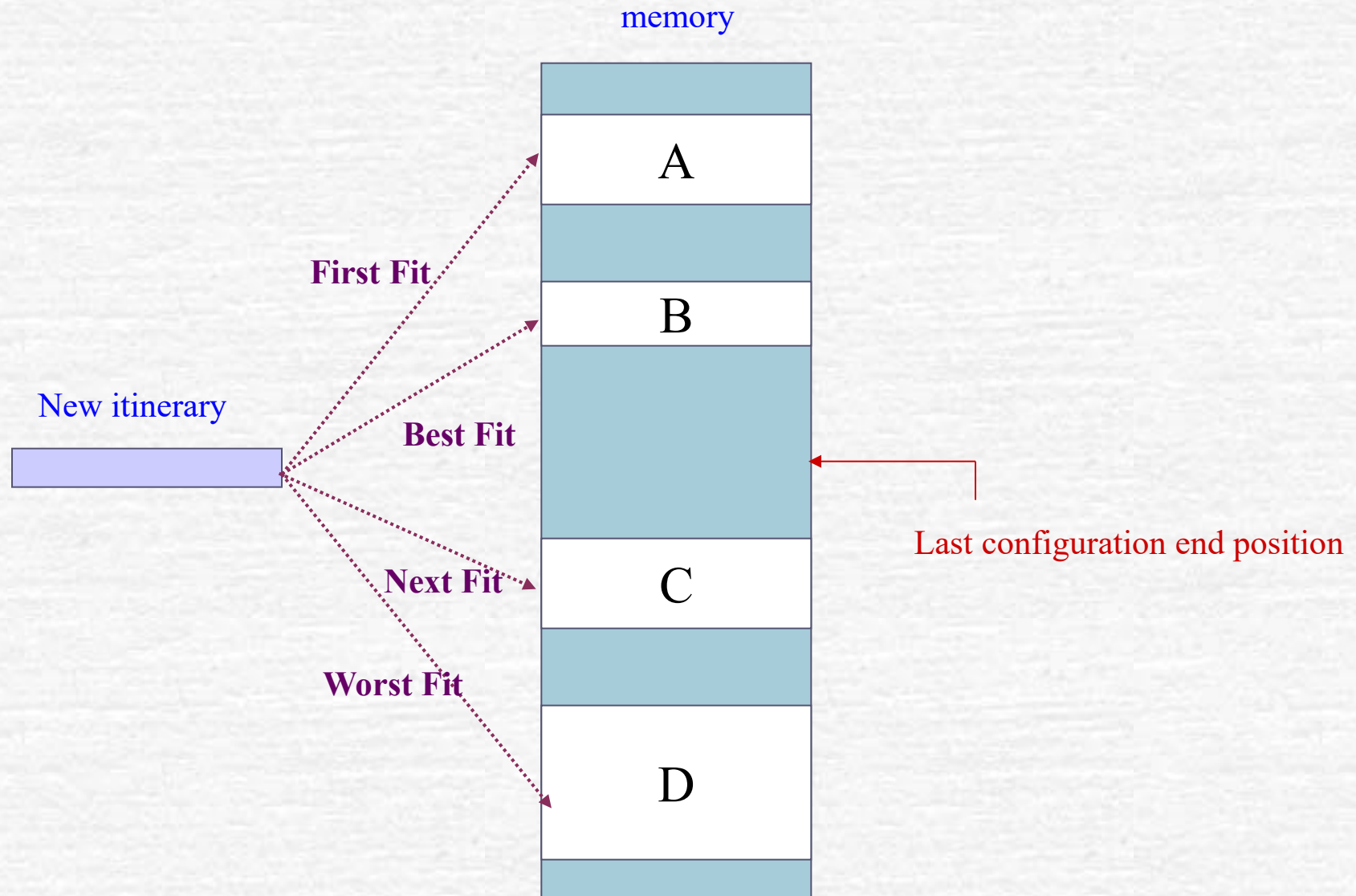
Figure 4-3 Example of external fragments



Partition selection algorithm

- **Correct the method:** Search for idle memory holes from the beginning of memory, and then place the process into the first hole that can accommodate it
- **Suboptimal method:** Continue your search down from where you last ended to improve your search time
- **Optimal method:** Find holes from all idle areas that are larger than this stroke and closest to this stroke
- **Least inadequate:** Allocate the largest hole to the stroke

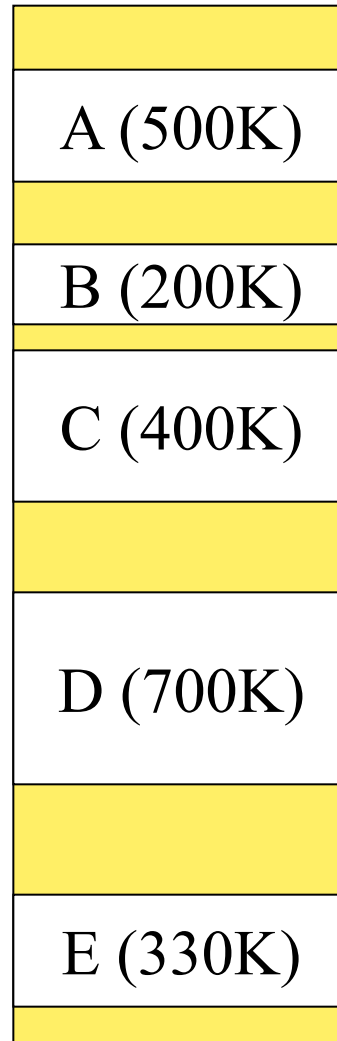
Figure 4-4 Example of partition selection algorithm



Classroom exercises

Q : Suppose the current allocation of memory is as shown in the figure, and the white area is the unused part. Please indicate when using the **first fit**, **the best fit**, **the next fit**, and **the worst fit method** :

1. Which zone will the new trip be assigned to?
2. What is the size of the internal fragments generated?



Last configuration end position

New itinerary

300K



Answer

algorithm	Select the area	Internal fragments
first law	A	200K
, the second law	C	100K
Optimal method	E	400K
The most unsuitable method	D	30K

Buddy system

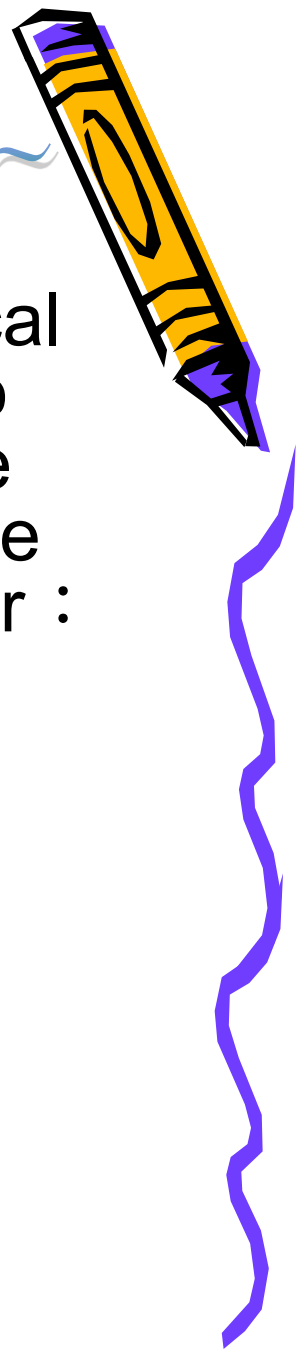
- Balance between **fixed** and **variable length** partitions
- Memory is allocated **in units of 2**
- When there is no hole of the appropriate size in memory, the larger hole is split in two, becoming two holes that are only half the size
- When the process ends, the **two partners merge into one** if the freed memory is of the same length as another adjacent hole

Figure 4-5 Example of the use of the Good Partner System

The original memory	block 1M					
Request A, 180K	A		256K		512K	
Request B, 60K	A		B 64K	128K	512K	
Request C, 250K	A		B 64K	128K	C	256K
Request D, 120K	A		B 64K	D	C	256K
release A	256K		B 64K	D	C	256K
release C	256K		B 64K	D	512K	
release B	256K		128K		D	512K
release D	1M					

Classroom exercises

範例補充包-Extra~

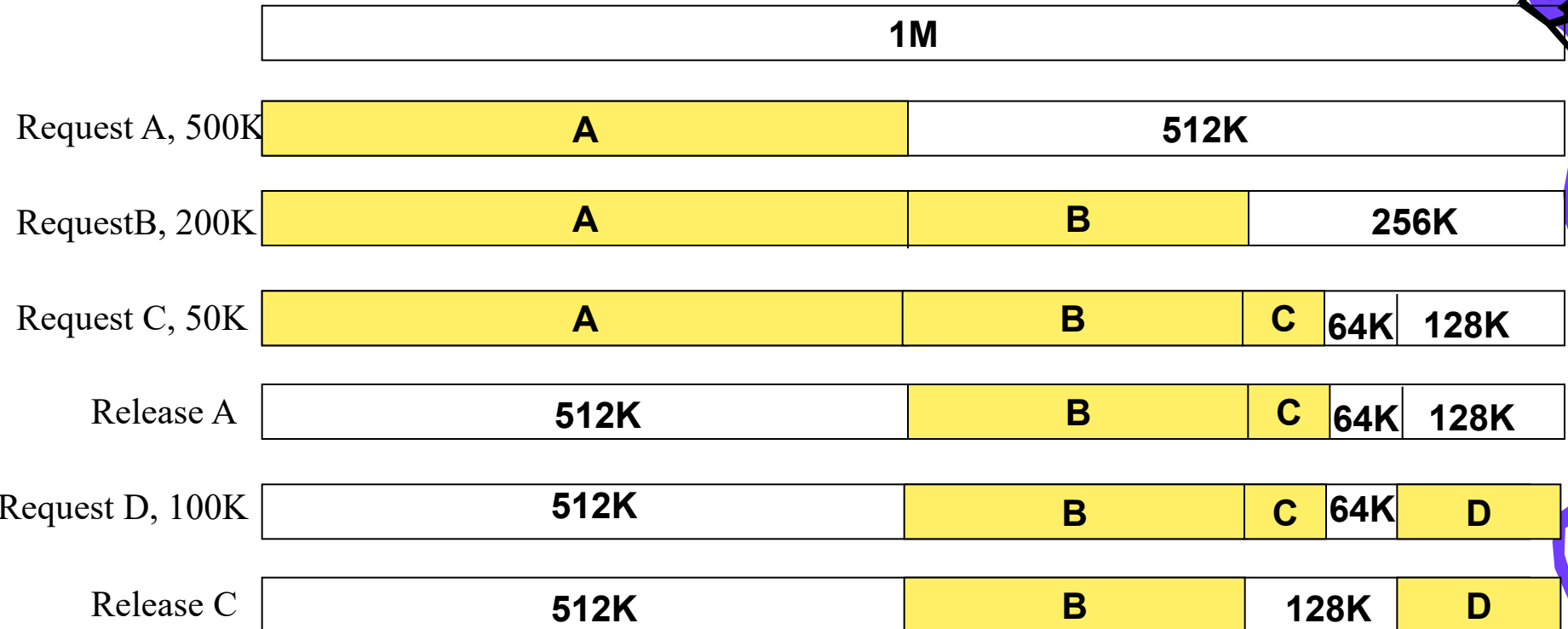
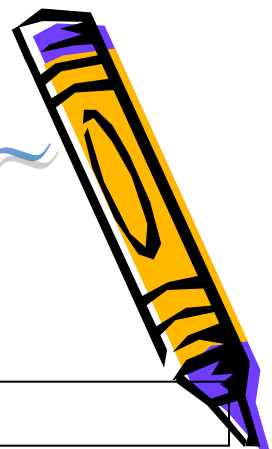


- Assuming that the system has 1M of physical memory and uses a good partner system to allocate memory areas, please describe the allocation of memory blocks according to the following memory request and release order :
 - Request A : 500K
 - Request B : 200K
 - Request C : 50K
 - Release A
 - Request D : 100K
 - Release C



Answer

範例補充色-Extra~



4-2 pagination

- Cutting the logical address space of the process into blocks of the same size is called a page
- The physical address space is also cut into units of the same size, called the **frame**
- Any page in the process can be placed in physical memory as long as it finds an unused page frame
- Each process is configured with a tab table that stores the **frame number of each page**

Figure 4-6 Example of pagination of logical and physical address spaces

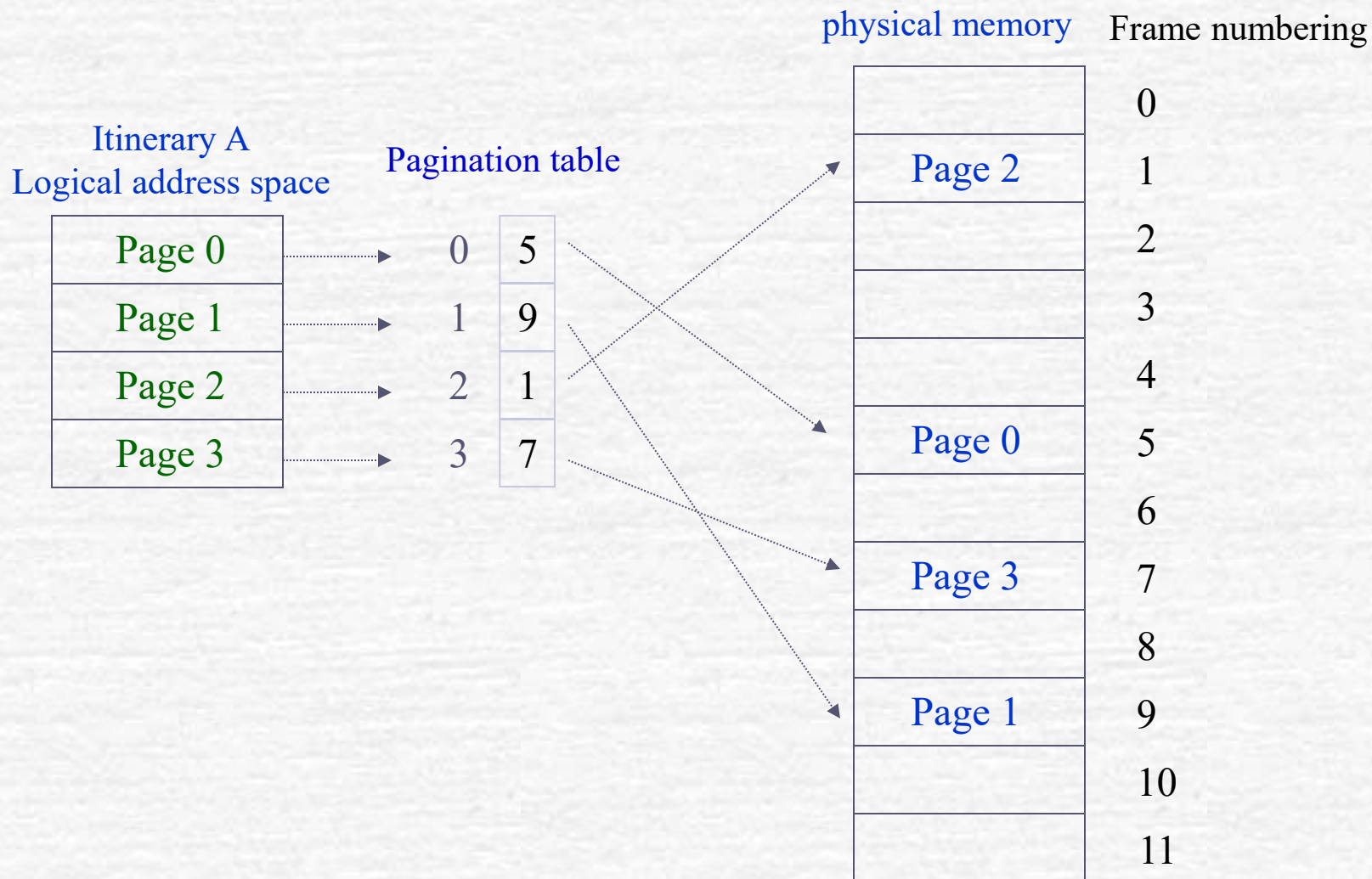


Figure 4-7 Using pagination to solve the external fragmentation problem

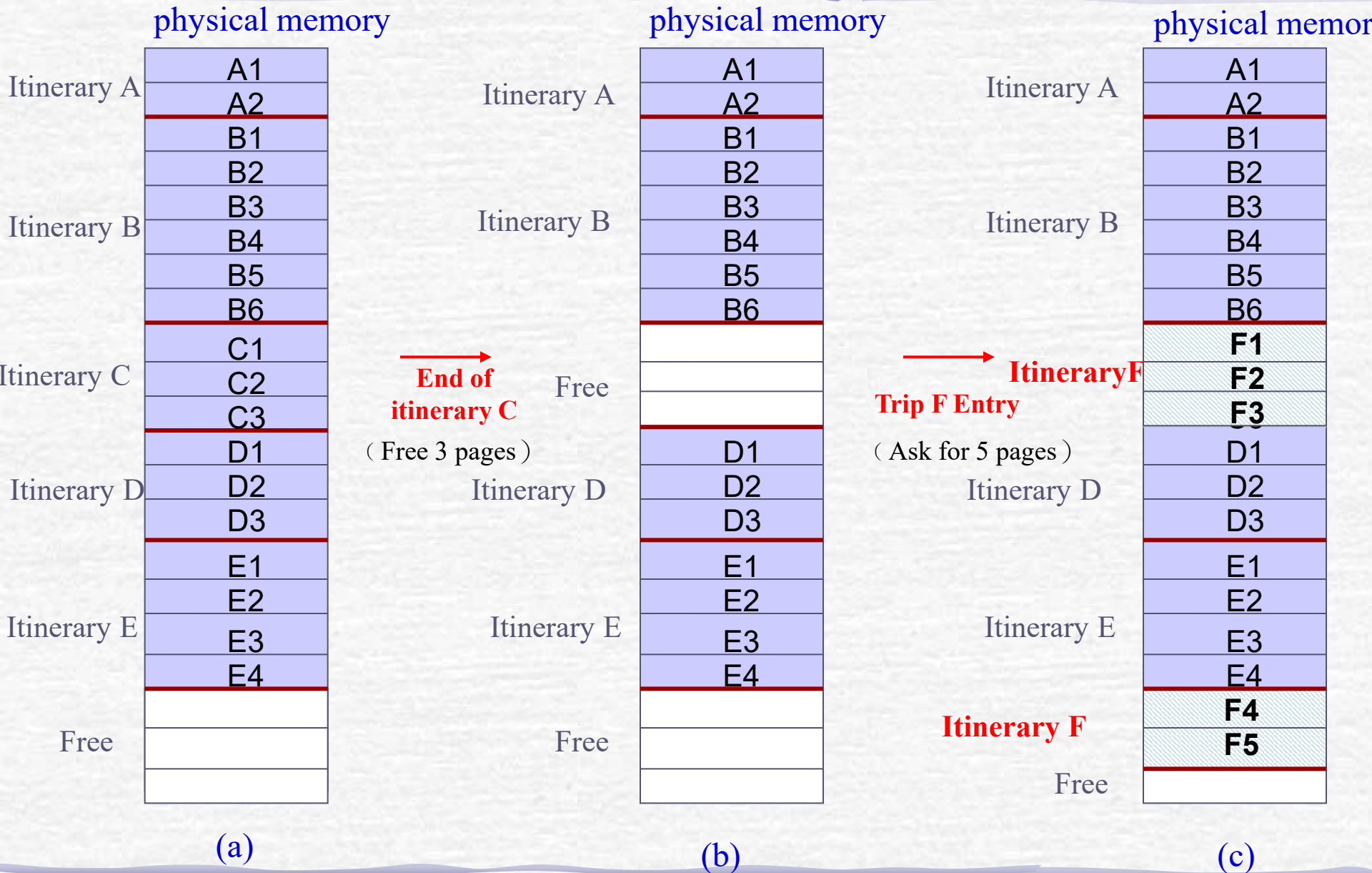
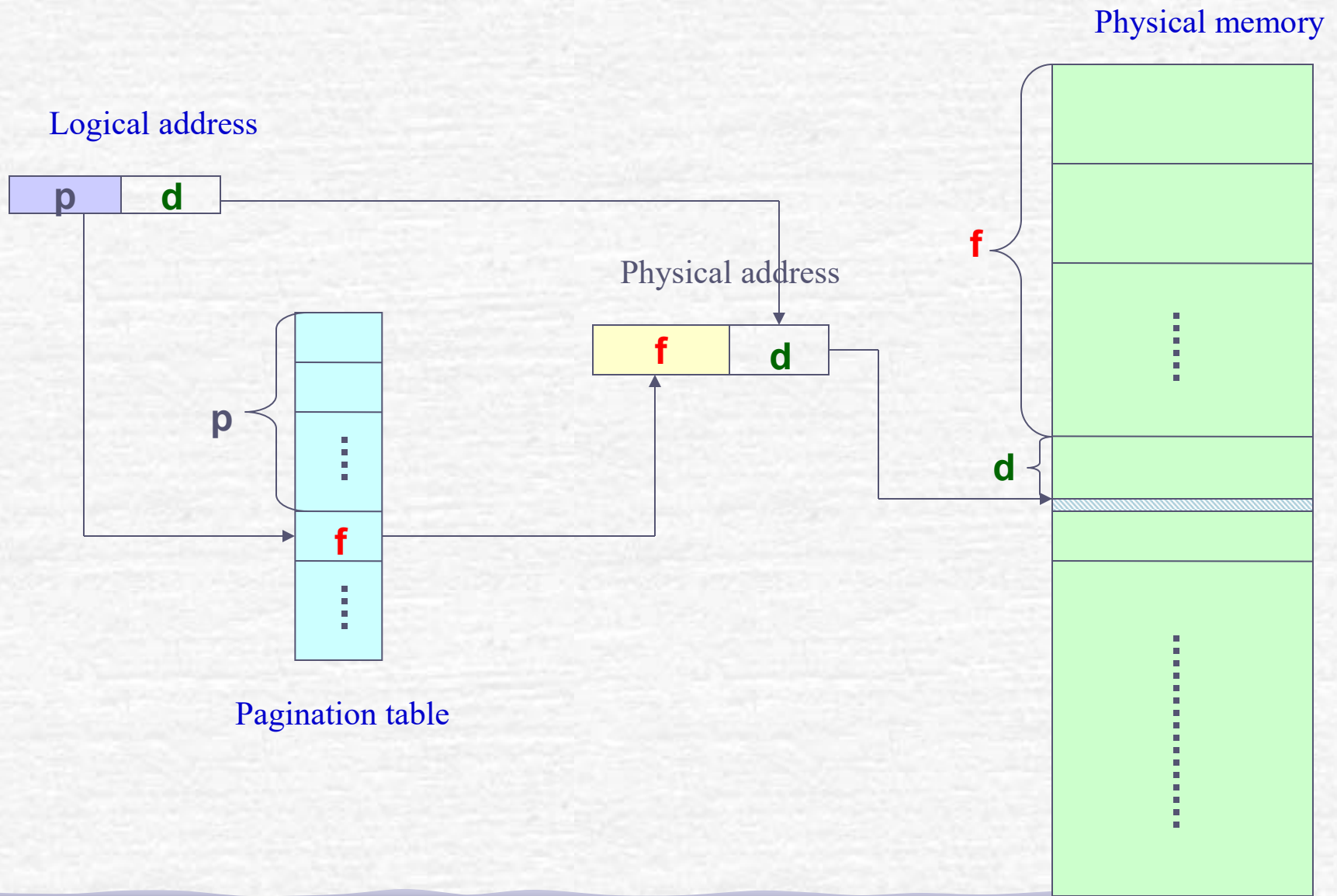
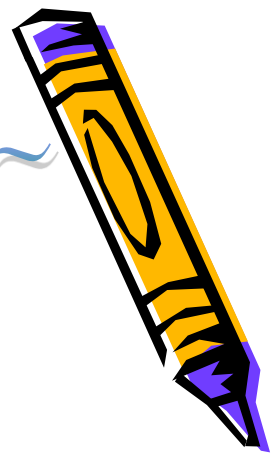


Figure 4-8 Address conversion during pagination





Classroom exercises

- Suppose there is a system with a logical address length of 32 and a page length of 4K :
 - How many bits does the page number and displacement occupy in a logical address?
 - What range should the value of the page number fall in?



Practice Answers



- The page length is 4K (=2¹²), so the displacement needs to be 12 bits
- The logical address length is 32, and the number of bits remaining is 20 (32-12), so the page number takes up 20 bits
- The range of page numbers from 0 to 2²⁰-1



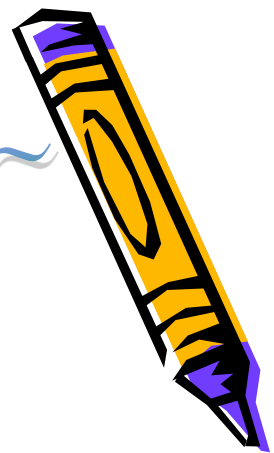
Classroom exercises

範例補充包 Extra~



- Assuming that the size of each page is 4K under a certain hardware architecture, please calculate the page number and offset amount of the following binary logical addresses (expressed in decimal characters)
- 0010 0000 0000 0000
 - 1010 0000 0001 0000





Problem solving

- Page number= $0010_2 = 2_{10}$
- Displacement = $0000\ 0000\ 0000_2 = 0_{10}$
- Page number= $1010_2 = 10_{10}$
- Displacement = $0000\ 0001\ 0000_2 = 16_{10}$



Fragmentation of the pagination design

- Using a pagination design avoids the problem of external fragmentation
- When the length of the stroke is not divisible by the length of the page, internal fragments are formed
- The average internal fragment is half the length of the page
- The smaller the page, the less waste it causes from internal fragmentation, but it puts a burden on storing and managing tab tables and reduces the efficiency of disk input/output

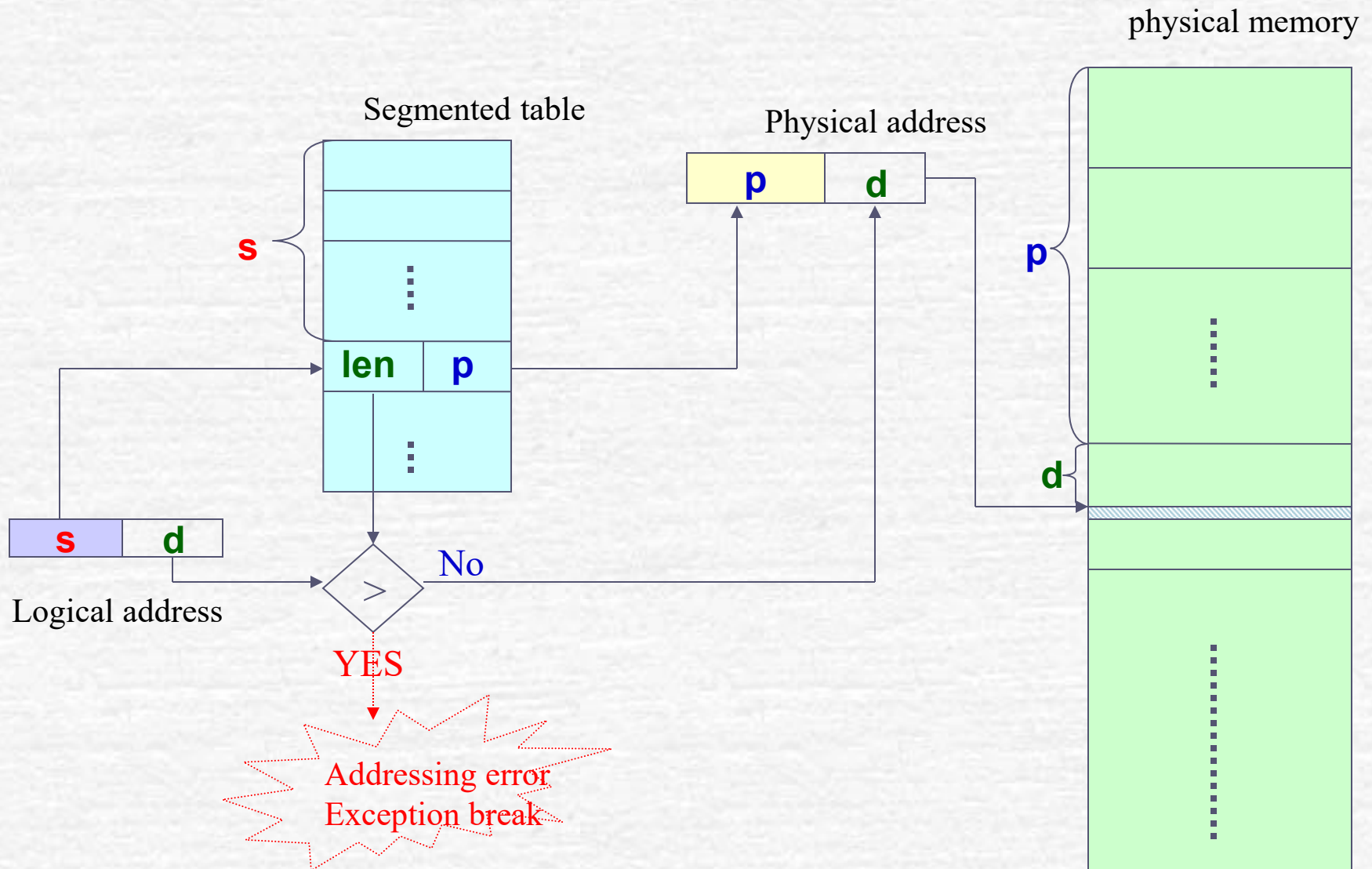
Implementation Discussion - A Buddy System for Linux

- Linux uses a good partner system under the paging mechanism to manage external fragmentation
- All unused frames are divided into 10 frame columns of different sizes, including 1, 2, 4, 8, 16, 32, 64, 128, 256, and 512 consecutive frames, each with a length of 4KB
- Based on the process's requirements for contiguous space, select contiguous blocks that are in the appropriate queue
- When the process releases unwanted frames, Linux attempts to join the partnerable areas and put them into a larger sequence

4-3 Segmentation

- Segment according to the characteristics of the process content, such as dividing the program into **program segments** and **data segments**
- Because segmentation involves the interpretation of the program, the responsibility falls on the shoulders of the programmer or compiler
- Paragraphs of varying lengths may be produced
- The operating system must manage a segmented table for each itinerary
 - The segment table contains the segment number, the amount of displacement, and the length of the segment

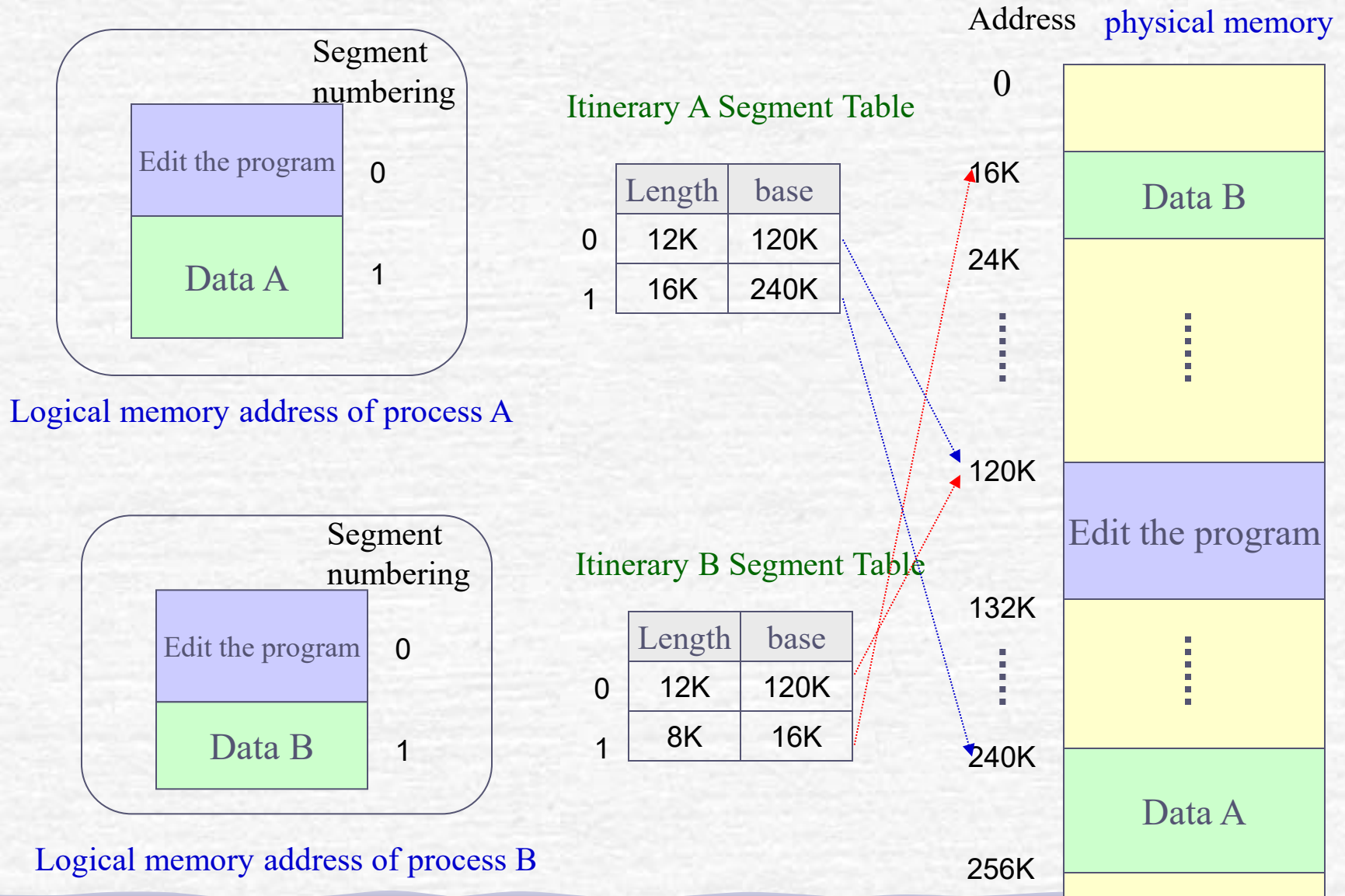
Figure 4-9 Address conversion during segmentation



Management of segmentation mechanisms

- Compare management of similar variable-length partitions
- Because each segment has different properties, a set of protection bits can be added to each item in the segmentation table to provide **segmentation protection** for each segmentation feature, such as **read-only** or **execution-only**
- It can achieve things like code (or libraries) or **data segment sharing**

Figure 4-10 Sharing of segments



Classroom exercises

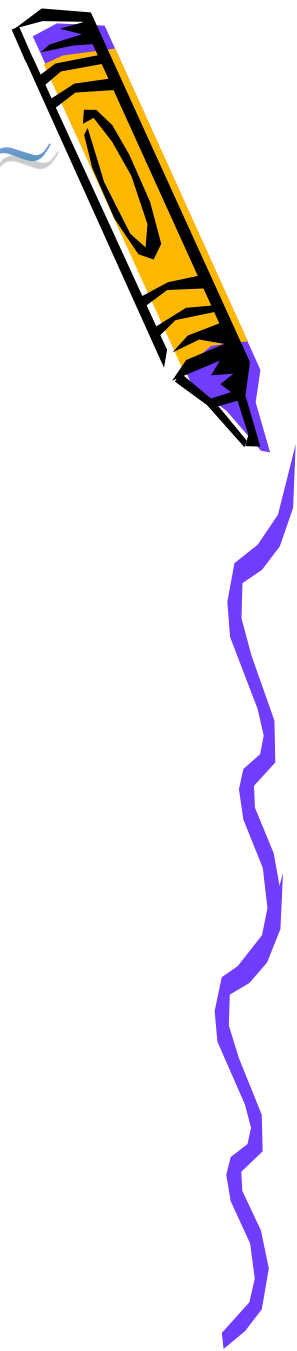
- Consider the contents of the following segmented table :

Segmentation	base	Length
0	512	512
1	2048	64
2	2560	128

Write the physical location corresponding to the following logical address (segment number, displacement). :

1. (0, 4)
2. (1, 12)
3. (1, 64)
4. (2, 110)





Answer

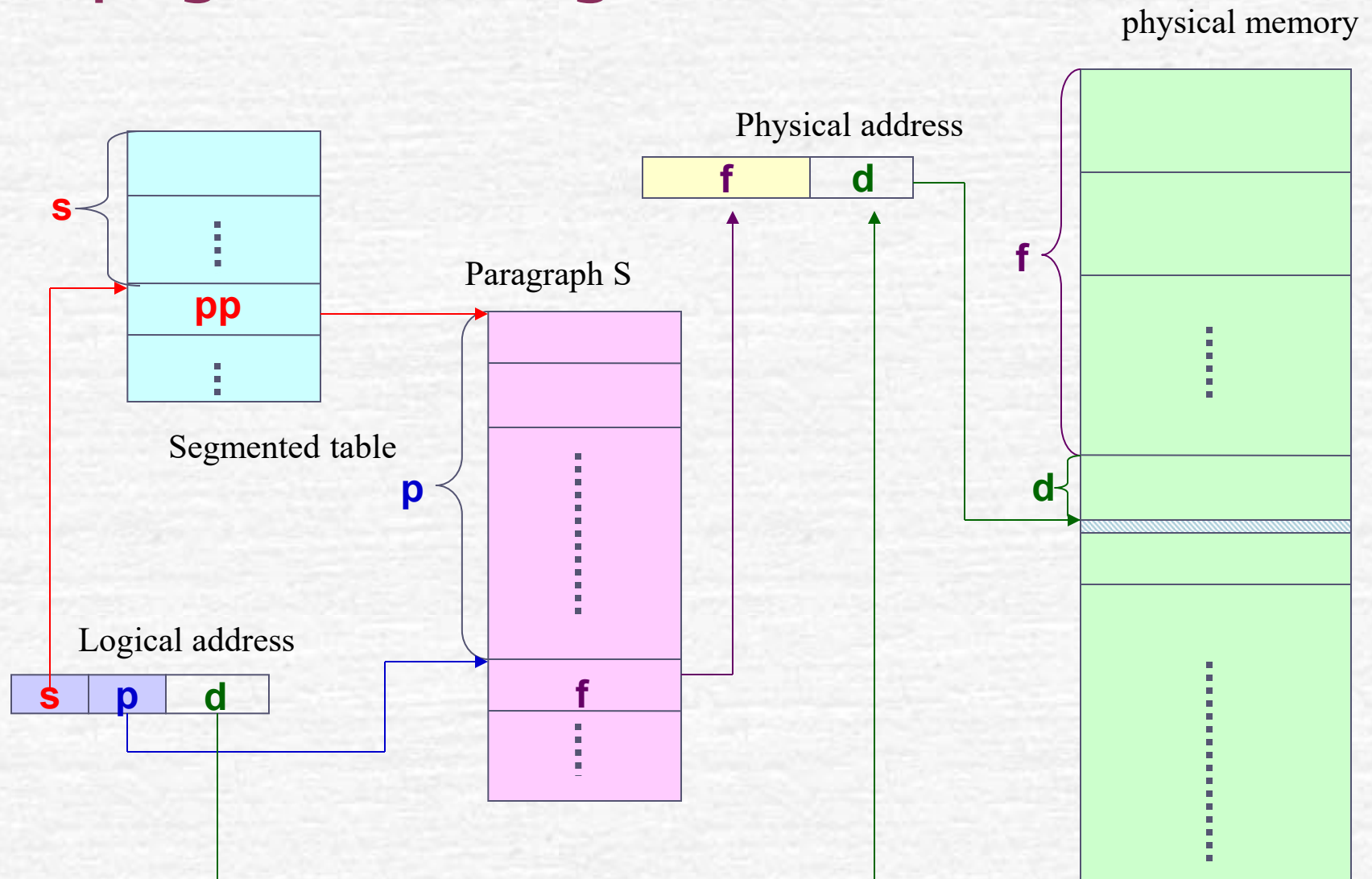
1. $512 + 4 = 516$
2. $2048 + 12 = 2060$
3. Exception break
4. $2560 + 110 = 2670$



Pagination segmentation

- The advantage of segmentation is to increase the flexibility of memory usage, but it also increases the complexity of memory management and causes external fragmentation problems
- Solution : Pagination is performed for each paragraph after segmentation, and each paragraph has its own pagination table
- There will be no **external fragments**, but there may be multiple **internal fragments** in one trip

Figure 4-11 Example of pagination segmentation



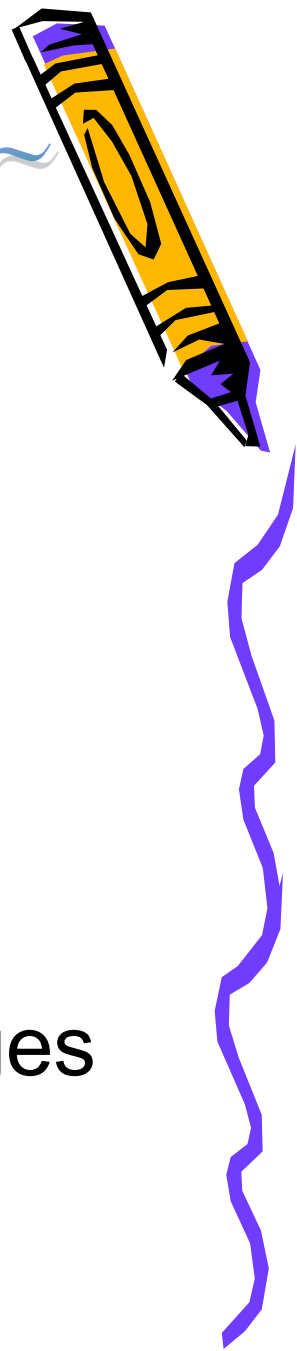
Classroom exercises

- Suppose that under a certain hardware architecture, a process can have up to **16** segments, each segment can have up to **128** pages, and each page size is 4K.
- Continuing from the previous question, assuming that the number of segments and page size remain unchanged, and the length of the logical address is 64 bits, how many pages can each paragraph have?



Practice Answers

範例補充包-Extra~



1. Segment numbering : 4位元

Paging number : 7位元

Displacement : 12位元

There are 23 bits in total

2. $64 - 4 - 12 = 48$

Each paragraph can have 2^{48} pages



Implementation Discussion - Types of Segmentation for Linux

- The reason why Linux adopts pagination segmentation
 - It is easier to provide the function of segmented sharing between trips
 - It can better adapt to the different segmentation support capabilities of each processor
- Types of segmentation in Linux : kernel program section, kernel data section, user program section, user data section, and LDT section, etc
- **LDT Segmentation** : Used to store the segmentation information of the process, which is equivalent to the segmentation table of the process

4-4 Replacement

- When the existing physical memory is filled with processes, new processes can no longer enter
- In order to increase the number of processes that can share the CPU, we can use the displacement operation to temporarily copy one or more processes **from memory to the secondary storage device**, and then move them back to memory to continue execution after a period of time
- It is up to the **operating system's mid-range scheduler** to decide which process to put in or out
- Normally, if you want to **swap** out a trip, it must be in a suspended state

Figure 4-12 Example of displacement operation

physical memory

